SET

Autonomous multi-agent orchestration for Claude Code

Give it a spec — get merged features.

April 2026

Agenda

Time	Topic
5 min	**The problem** — Why "prompt and pray" isn't enough
10 min	**Six pillars** — SPECIFY, DECOMPOSE, EXECUTE, SUPERVISE, VERIFY, LEARN
5 min	**The input** — Spec + Design = quality input
15 min	**E2E Demo: MiniShop** — Walking through the full pipeline
5 min	**Works everywhere** — Greenfield, brownfield, isolated unit
5 min	**Dashboard & Monitoring** — Real-time visibility
5 min	**Lessons learned** — 100+ runs in production
5 min	**Roadmap** — Where we're heading

1. The problem

Why isn't prompting enough?

AI coding today — the challenges

Problem	Typical approach	Result
Divergence	Run prompt twice = two different results	Not reproducible
Hallucination	Agents make up what they don't know	Missing features
Quality roulette	LLM judges code quality	Inconsistent
Spec drift	"Tests pass" ≠ "spec is satisfied"	Partial implementation
Amnesia	Every session starts from scratch	Repeated mistakes
Bug fixing	Manual debugging, hours	Lost time

> Most AI coding tools are **non-deterministic** — same prompt, different result. SET treats this as an engineering problem.

How SET addresses it

Challenge	SET solution	Result
Divergence	3-layer template system	**83-87%** structural convergence
Hallucination	OpenSpec workflow + acceptance criteria	Implements against the spec
Quality roulette	Programmatic gates (exit codes, not LLM judgment)	Deterministic pass/fail
Spec drift	Coverage tracking + auto-replan	100% spec coverage
Amnesia	Hook-driven memory (5 layers)	100% context capture
Bug fixing	Issue pipeline: detect → investigate → fix	**30 second** recovery

> **"We don't prompt — we specify."**

2. Six pillars

The architecture behind it

SPECIFY > DECOMPOSE > EXECUTE > SUPERVISE > VERIFY > LEARN

SPECIFY	DECOMPOSE	EXECUTE	SUPERVISE	VERIFY	LEARN
Structured input, not prompts	Intelli-gent plan, not guessing	Structured implem., not free rein	Three-tier supervis., not baby-sitting	Determ. quality, not LLM vibes	Every run improves the next

Every feature, gate, and automation maps to one of these six pillars.

Change lifecycle — state machine

```
                  ┌──────────────────────────────────────────────┐
                  │              SPECIFY + DECOMPOSE             │
spec.md ────────▶ │  digest ──▶ triage ──▶ plan (DAG) ──▶ dispatch │
                  └──────────────────────────────────────────────┘
                                        │
                                        ▼
                  ┌──────────────────────────────────────────────┐
                  │              EXECUTE (per change)            │
                  │  pending ──▶ dispatched ──▶ running          │
                  │                             │                │
                  │              Ralph Loop:    │                │
                  │              proposal ──▶ design ──▶ spec    │
                  │              ──▶ tasks ──▶ code ──▶ test     │
                  └──────────────────────────────────────────────┘
                                        │
                                        ▼
                  ┌──────────────────────────────────────────────┐
                  │              VERIFY                          │
                  │  test ──▶ build ──▶ E2E ──▶ review           │
                  │  ──▶ spec_coverage ──▶ smoke                 │
                  │        │                                     │
                  │        ├─ PASS ──▶ merge_queue               │
                  │        └─ FAIL ──▶ agent fixes ──▶ VERIFY again │
                  └──────────────────────────────────────────────┘
                                        │
                                        ▼
                  ┌──────────────────────────────────────────────┐
                  │              SUPERVISE + LEARN               │
                  │  merge (FF-only) ──▶ post-merge smoke        │
                  │  ──▶ coverage check                          │
                  │        │                                     │
                  │        ├─ 100% ──▶ done                      │
                  │        └─ <100% ──▶ REPLAN ──▶ new DECOMPOSE │
```

Gradual escalation — when things break

```
Normal operation
      │
      ▼
 ┌─ L1: Warning ──────────────── Log + notification, no intervention
 │
 ├─ L2: Restart ──────────────── Agent restart with context pruning (~70% success)
 │
 ├─ L3: Redispatch ─────────────── Full worktree rebuild (max 2x)
 │
 └─ L4: Give up ──────────────── Change → failed (better than burning tokens)
```

The watchdog is **context-aware**:

- `pnpm install` 90s without stdout? → Grace period (not a stall)
- Prisma migration running? → Extended timeout

3. The input

Output quality depends on input quality

What a good spec looks like

```
spec.md
├── Data model          -- entities, fields, relations, enums
├── Pages               -- sections, columns, components
├── Design tokens       -- hex colors, fonts, spacing values
├── Auth & roles        -- protected routes, registration
├── Seed data           -- realistic names, not "Product 1"
├── i18n                -- locales, URL structure
└── Business reqs       -- user stories, acceptance criteria
```

You are the product owner, the agents are the dev team, the spec is the sprint backlog.

The difference: this sprint takes hours, not weeks.

Spec + Figma Design

```
$ head -35 tests/e2e/scaffolds/minishop/docs/v1-minishop.md

# MiniShop v1 — Webshop Feature Spec

> Next.js 14+ App Router webshop with Prisma (SQLite), shadcn/ui, Tailwind CSS, NextAuth.js v5

## Design

**Figma Design:** https://www.figma.com/make/9PH3uS4vWjSj6cUPhTGZSt/set-minishop?p=f&t=zvhTdumJeYUpKrJm-0

**Local design snapshot:** `docs/figma-raw/9PH3uS4vWjSj6cUPhTGZSt/` — pre-fetched via `set-figma-fetch`, contains

## Starting Point

There is NO application code in the scaffold. Agents create everything from scratch.

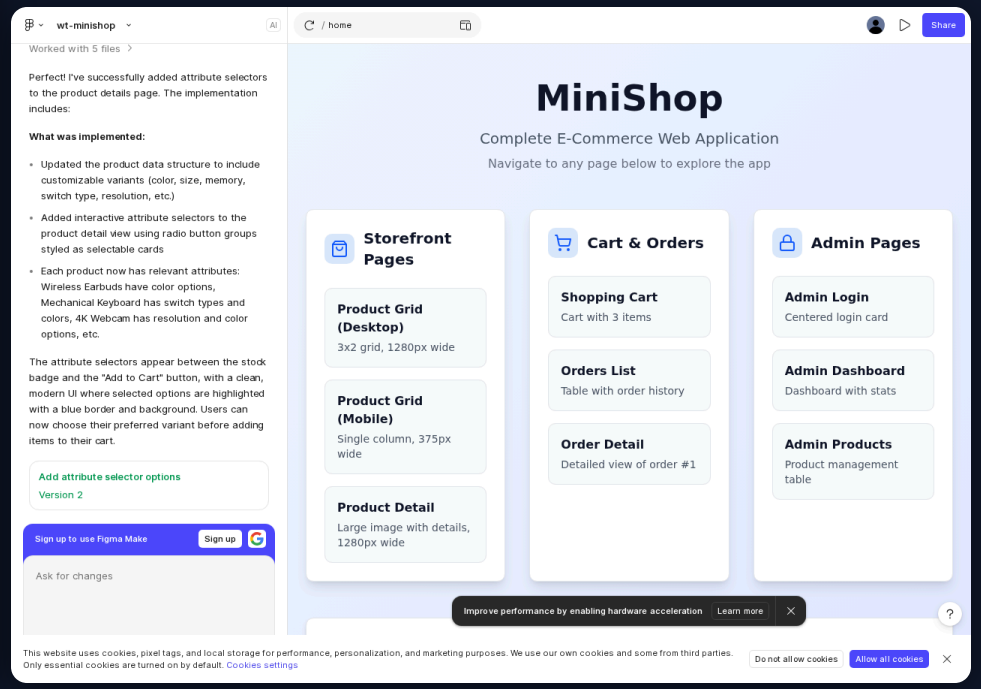
Platform configs are deployed by `set-project init --project-type web --template nextjs` before orchestration sta

**Setup (done by `run.sh` before orchestration):**
1. Copy this spec to `docs/v1-minishop.md`
2. `git init && set-project init --project-type web --template nextjs`
3. Orchestration starts — agents create everything from this spec

## Dependencies (package.json)

Agents must create `package.json` with these dependencies:

```jsonc
{
  "name": "minishop",
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "test": "vitest run",
    "test:e2e": "playwright test"
  },
```
```



Left: Markdown spec — the business requirements

Right: Figma Make design — the visual blueprint

4. E2E Demo: MiniShop

Building a webshop from a spec — live

MiniShop — the task

A **Next.js 14 e-commerce application** — built entirely from a spec:

- Product listing and detail pages
- Shopping cart (add, remove, update quantity)
- Checkout and order management
- Admin authentication (login, registration)
- Admin CRUD (product management)
- Seeded database with realistic data

Question: How long does it take, how many bugs, how much human intervention?

The pipeline

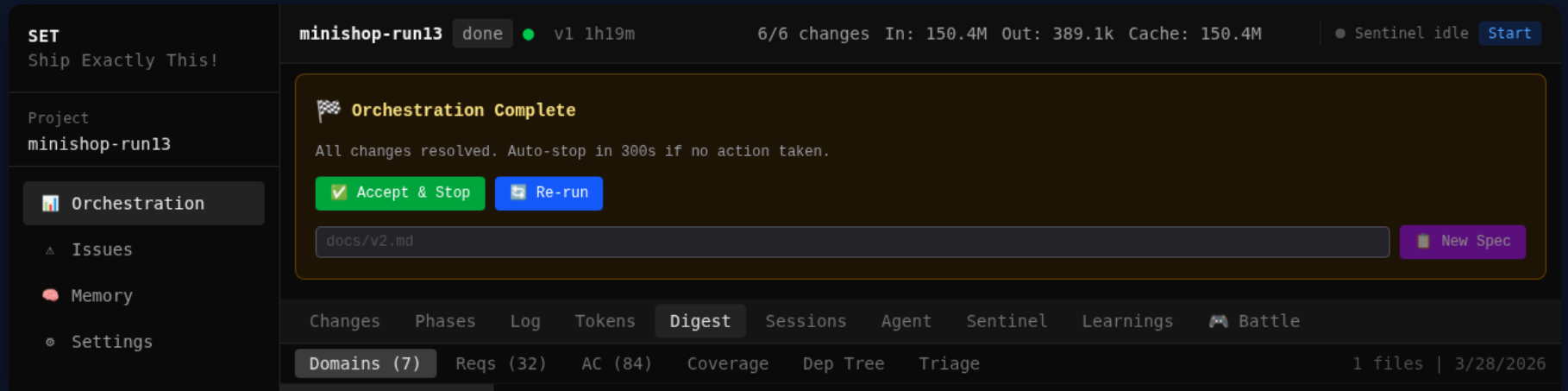
```
spec.md ──> digest ──> decompose ──> parallel agents ──> verify ──> merge ──> done
```

```
spec.md + design-snapshot.md (Figma)
  │
  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ Sentinel (autonomous supervisor)                                         │
│  ├─ digest: spec → requirements + domain summaries                       │
│  ├─ decompose: independent changes (DAG)                                 │
│  ├─ dispatch: into git worktrees in parallel                             │
│  ├─ monitor: 15s polling, crash recovery                                 │
│  ├─ verify: quality gates per change                                     │
│  ├─ merge: FF-only + post-merge smoke                                    │
│  └─ replan: until spec is 100% covered                                   │
└──────────────────────────────────────────────────────────────────────────┘
```

Step 1: Digest — Extracting requirements

The digest module extracts from the spec:

- **Domains** (e.g., Products, Cart, Auth, Admin)
- **Requirements** per domain (e.g., 32 reqs)
- **Acceptance criteria** (e.g., 84 WHEN/THEN)



Step 2: Decompose — Independent changes

The planner creates **independently implementable changes** from requirements:

Change	Depends on	Size
`project-infrastructure`	--	Base structure, config
`products-page`	infrastructure	Product list + details
`cart-feature`	products	Cart functionality
`admin-auth`	infrastructure	Admin login
`orders-checkout`	cart, admin-auth	Orders + checkout
`admin-products`	admin-auth, products	Admin CRUD

Dependency DAG — broken into phases:

Step 3: Dispatch — Parallel agents

Each change gets its own **git worktree** — an isolated dev environment:

```
main/
├── .worktrees/
│   ├── project-infrastructure/   ← Agent 1
│   ├── products-page/            ← Agent 2
│   ├── admin-auth/               ← Agent 3 (parallel with #2!)
│   ├── cart-feature/             ← Agent 4
│   ├── orders-checkout/          ← Agent 5
│   └── admin-products/           ← Agent 6
```

Inside each agent the **Ralph Loop** runs:

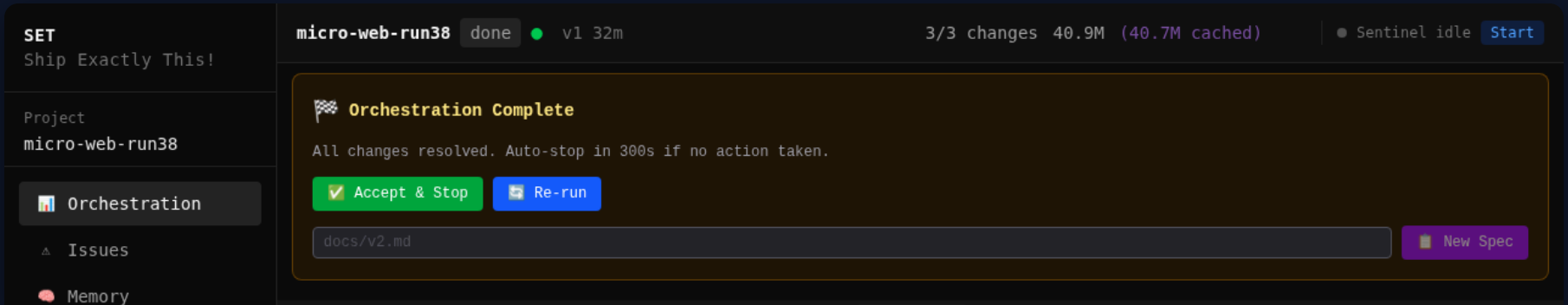
```
proposal → design → spec → tasks → implementation → tests → done
```

> One agent = one change = one worktree = full isolation.

Step 4: Monitor — Real-time supervision

The Sentinel polls every agent **every 15 seconds**:

- **Progress tracking** — where is each change?
- **Stall detection** — no progress for >120s → investigation
- **Crash recovery** — PID gone? → diagnose → restart
- **Token tracking** — cost monitoring per change



Meanwhile the agent works...

SET
Ship Exactly This!

Project
micro-web-run38

- Orchestration
- Issues
- Memory
- Settings

Manager: running

micro-web-run38 done v1 32m 3/3 changes 40.9M (40.7M cached) Sentinel idle Start

Orchestration Complete

All changes resolved. Auto-stop in 300s if no action taken.

Accept & Stop | Re-run

docs/v2.md | New Spec

Changes | Phases | Log | Tokens | Context | Digest | Sessions | Agent | Sentinel | Learnings | Battle

Agent Chat New Session

Köszönj és add meg a rövid orchestration státusz összefoglalót.

Szia!

Az orchestration befejeződött. Rövid összefoglaló:

Állapot: Kész (done)
3/3 change sikeresen merge-elve

Change	Státusz	Tokenek
foundation-navigation	merged	51,967
content-pages	merged	32,531
contact-form	merged	34,089

Összes token: 118,587 (mind opus modellel)

Type a message or use voice input... Send

Step 5: Verify — Quality Gates

Each change passes **6 gates** before merge:

```
Jest/Vitest (8s) → Build (35s) → Playwright E2E (45s)
    → Code Review (25s) → Spec Coverage → Post-merge Smoke (15s)
```

Gate	Time	Checks	Type
Test	8s	Unit/integration tests	Deterministic (exit code)
Build	35s	Type check + bundle	Deterministic (exit code)
E2E	45s	Browser-based tests	Deterministic (exit code)
Review	25s	Code quality	LLM (CRITICAL = fail)
Spec Coverage	--	Requirement coverage	LLM + pattern match
Smoke	15s	Post-merge sanity check	Deterministic

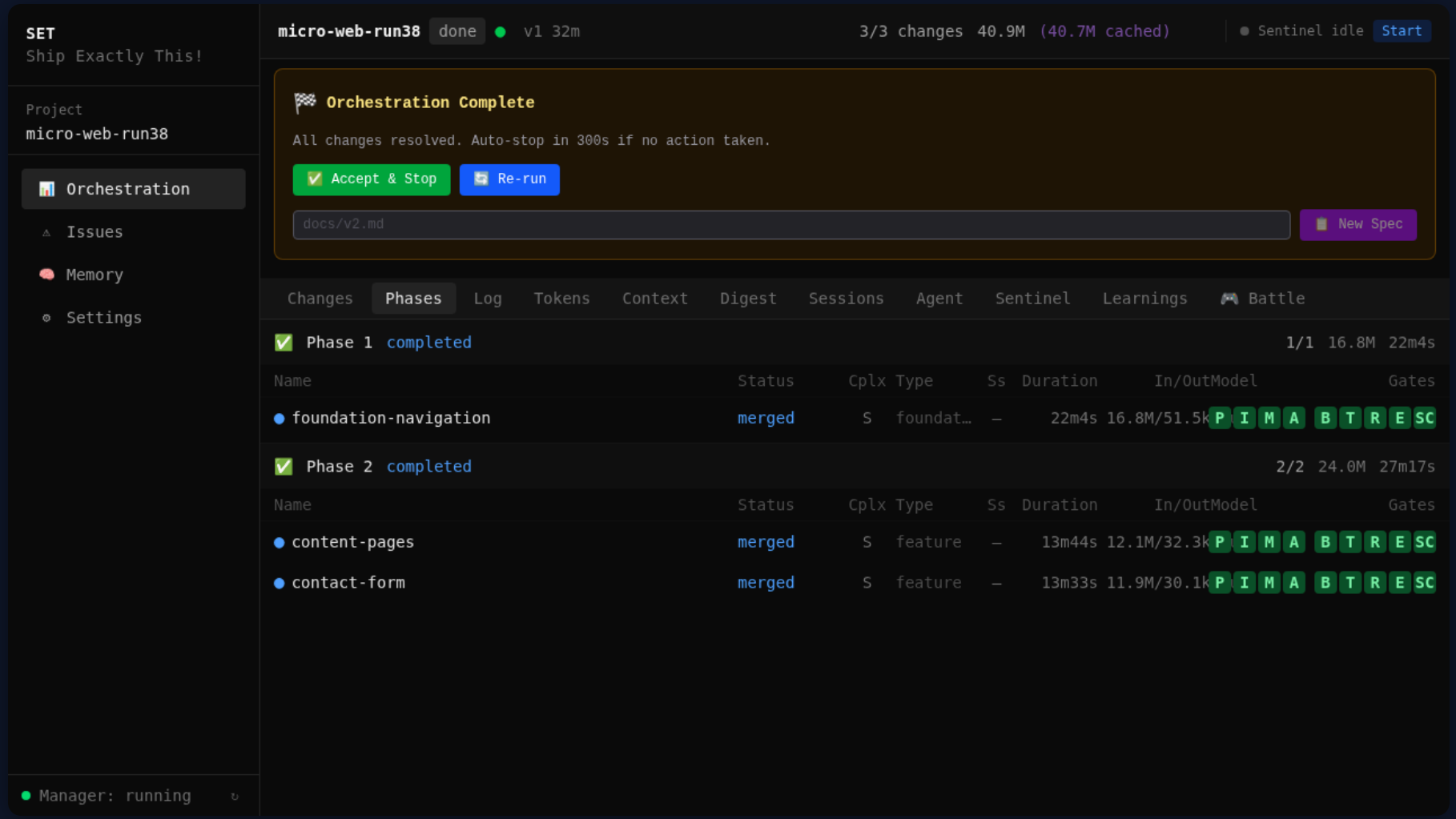
Self-Healing: 5 gate failures, 5 automatic fixes

The MiniShop run had **5 gate failures** — all auto-fixed:

#	Failure	Gate	Fix
1	Missing test file	Test	Agent added the test file
2	Jest config error	Build	Agent fixed the path mapping
3	Playwright auth tests (3 specs)	E2E	Agent updated the redirects
4	Post-merge type error	Build	Agent synced with main
5	Cart test race condition	E2E	Agent added `waitForSelector`

> Without gates, these **5 bugs would have merged into main** and caused

Step 6: Merge — Integration



Step 7: Replan — 100% coverage

```
                    ┌──── coverage < 100%? ────┐
                    │                          │
merge ──▶ coverage check ──▶ done          replan
                                              │
                                     decompose again
                                     for missing REQs
```

The replan gate checks:

- Has every `[REQ: ...]` been implemented?
- Are there uncovered domains?
- If yes: **automatic planning of new changes** for the gaps

> The system doesn't stop until the spec is 100% covered.

Works everywhere

Not just "build me an app from scratch"

Greenfield, Brownfield, Isolated Unit

Mode	What it means	Example
Greenfield	Full app from spec + design	MiniShop: 6/6 merged, 1h 45m, 0 interventions
Brownfield	Existing codebase + new features	SET builds itself: 1,500+ commits, 376 specs
Isolated unit	One module, one feature, one fix	"3 API endpoints with auth in my existing Next.js"

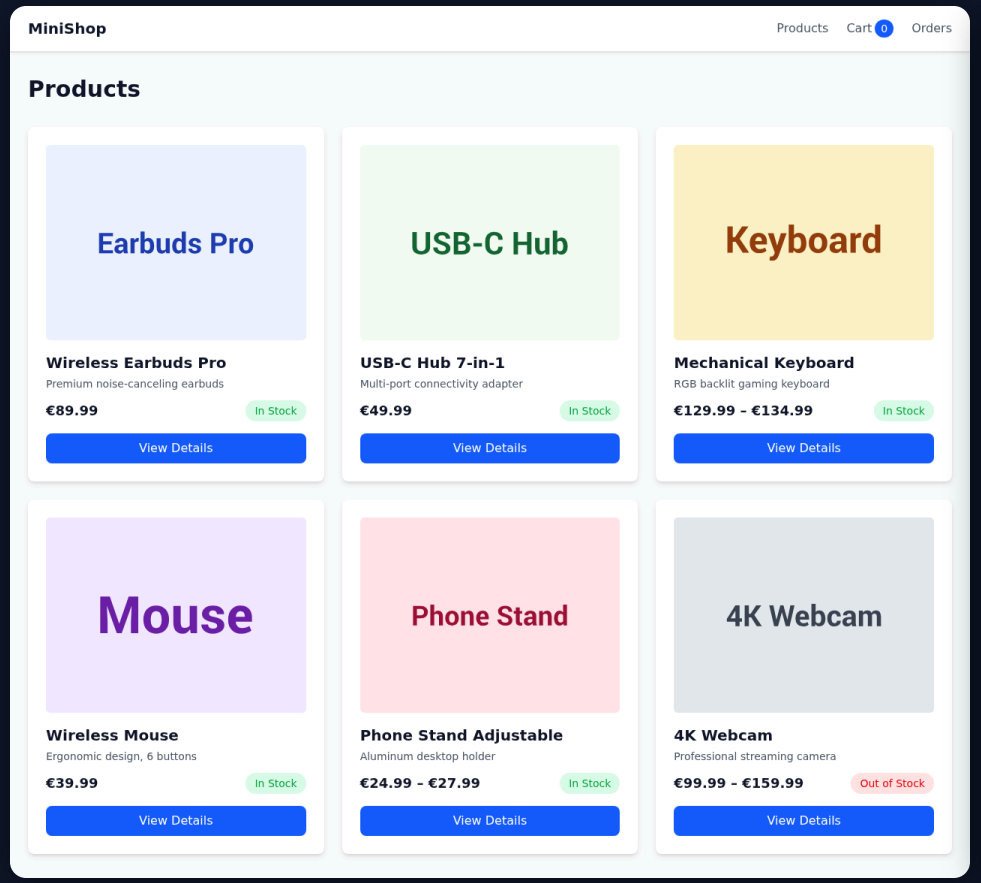
The pipeline is **the same** in all three modes:

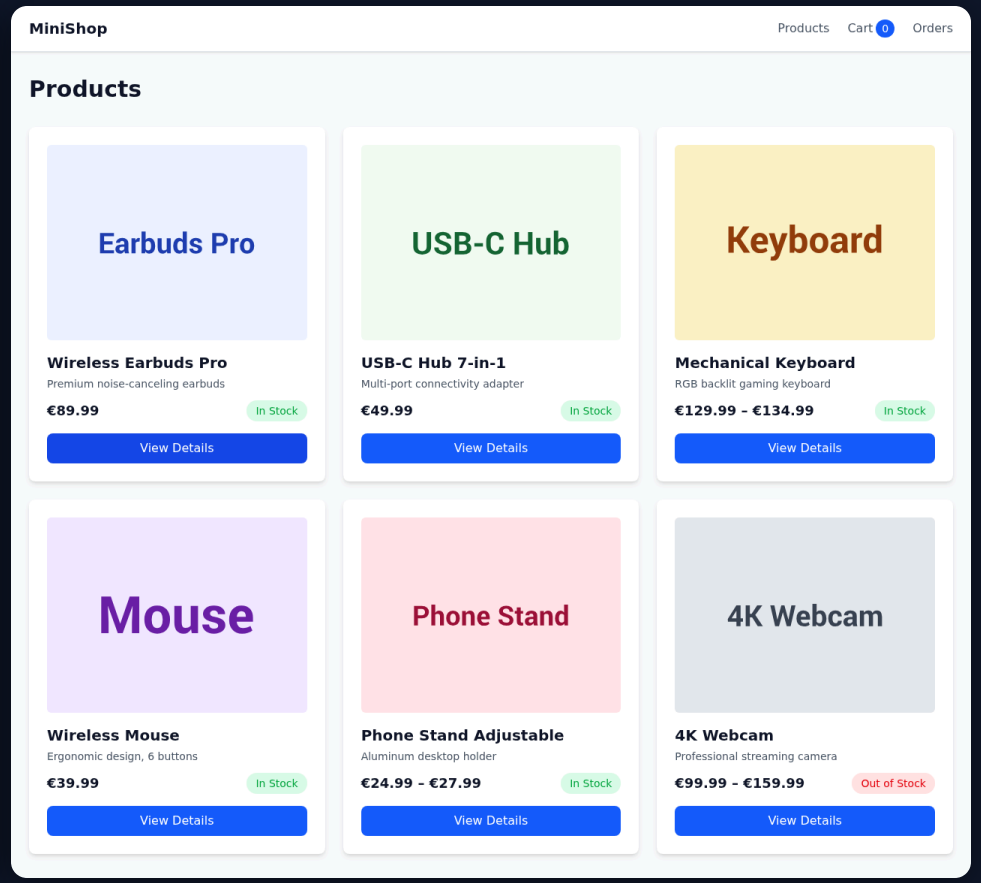
- Worktree isolation (even with 1 change)
- Quality gates (runs your existing tests too)
- The system **reads existing code** before doing anything

The result

What the pipeline produced

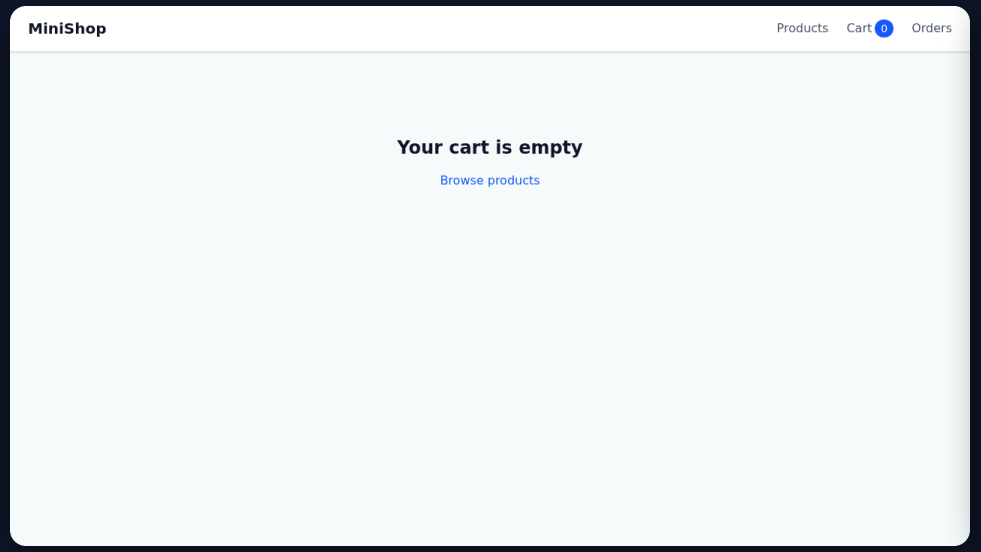
MiniShop — the finished application

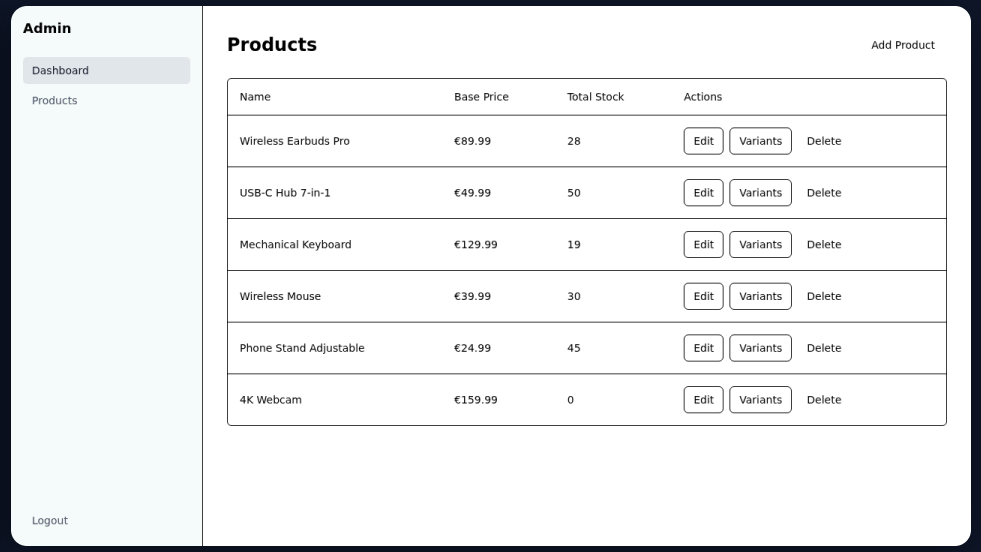




Product listing and **product details** — real data, working navigation.

MiniShop — Cart and Admin

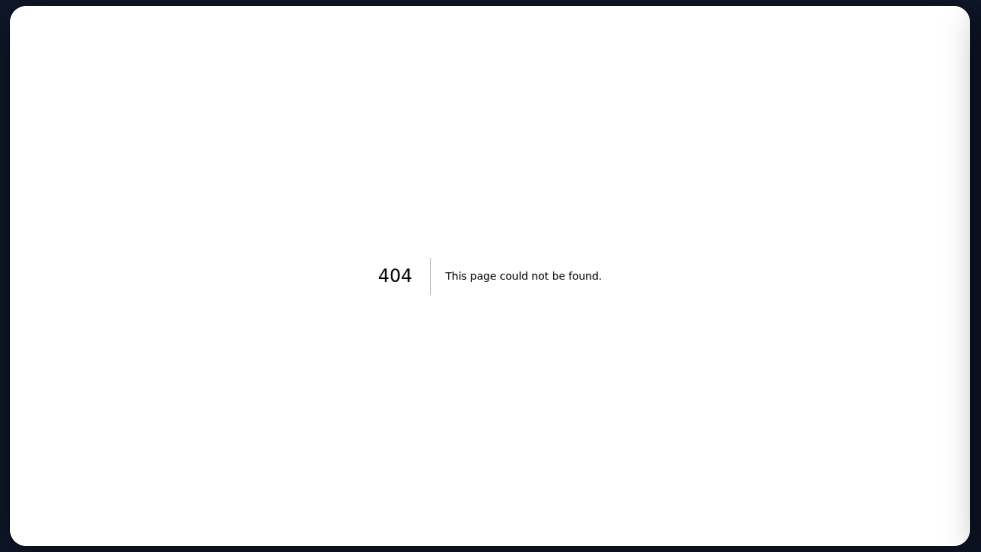


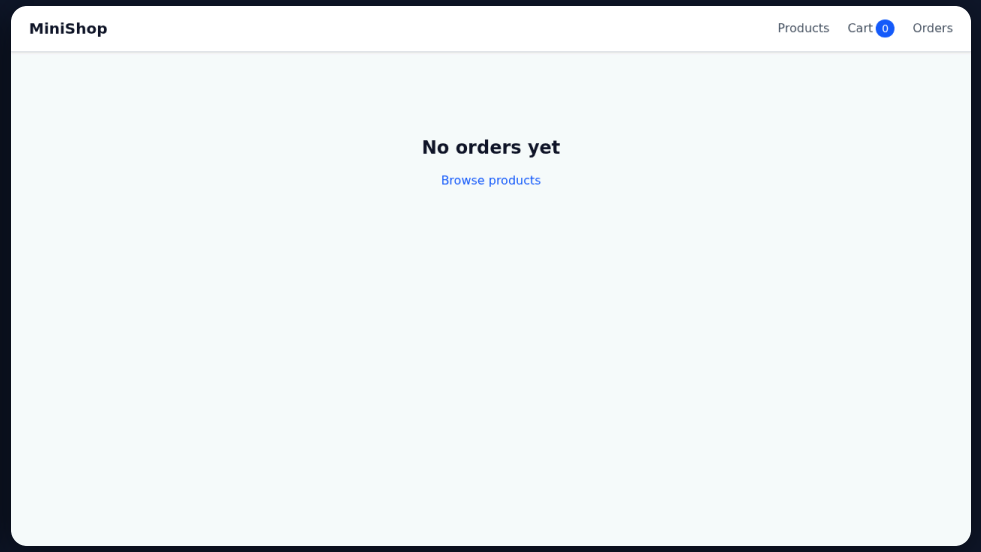


Left: Working cart — quantity update, totals

Right: Admin dashboard — protected route, session-based auth

MiniShop — Admin CRUD and Orders





Left: Product CRUD (create, edit, delete)

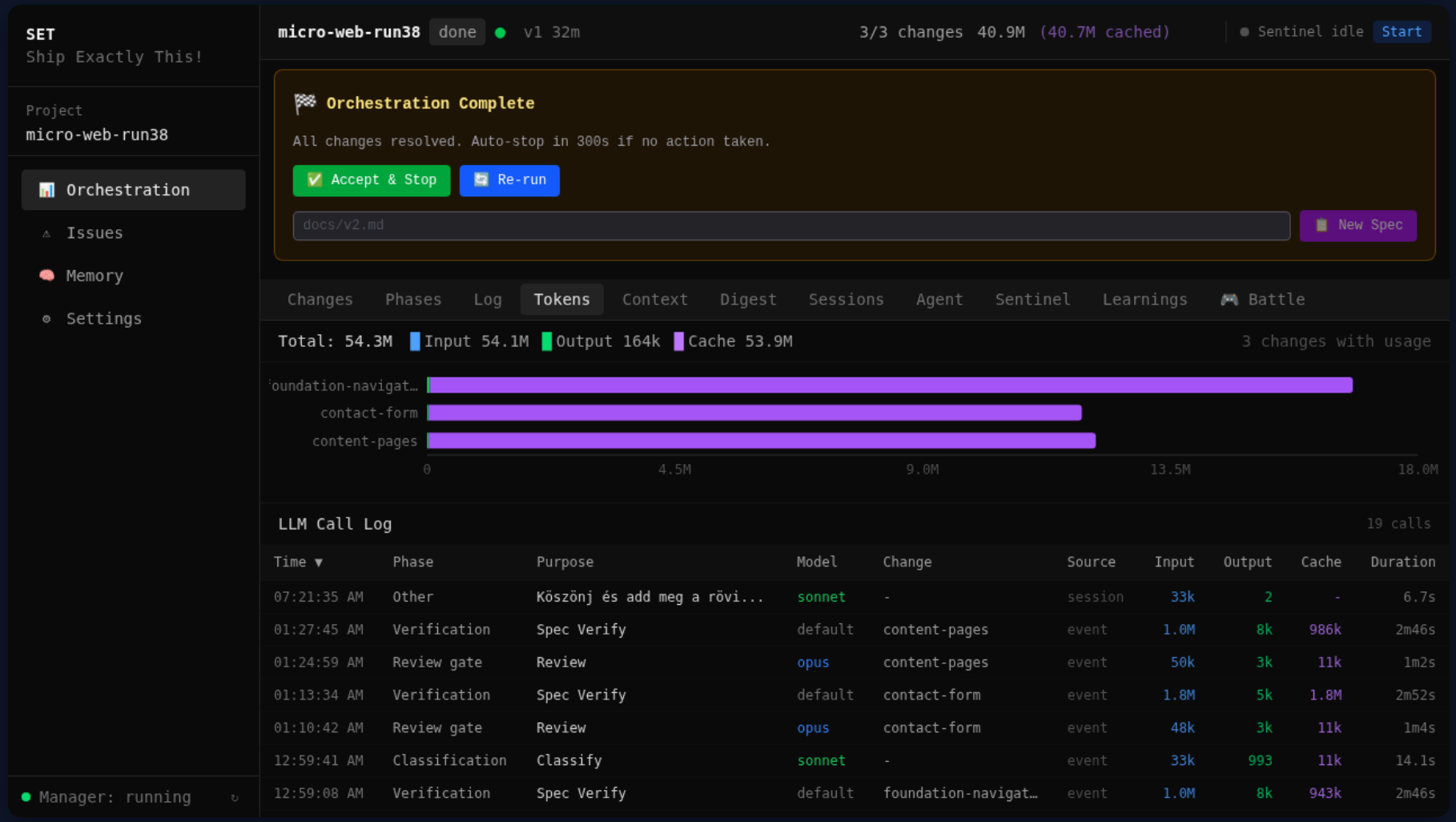
Right: Order management — statuses, filtering

> Every page has a **working database**, **functional navigation**, and **responsive**

The numbers

Metric	Value
Planned changes	**6**
Successfully merged	**6/6 (100%)**
Total wall time	**1h 45m**
Active build time	~1h 25m
Human interventions	**0**
Merge conflicts	**0**
Jest unit tests	38 (6 suites)
Playwright E2E tests	32 (6 spec files)
Git commits	39
Total tokens	2.7M

Token usage


SET
Ship Exactly This!
Project
micro-web-run38
Orchestration
Issues
Memory
Settings
Manager: running
micro-web-run38 done v1 32m
3/3 changes 40.9M (40.7M cached)
Sentinel idle Start
Orchestration Complete
All changes resolved. Auto-stop in 300s if no action taken.
Accept & Stop
Re-run
docs/v2.md
New Spec
Changes Phases Log Tokens Context Digest Sessions Agent Sentinel Learnings Battle
Total: 54.3M Input 54.1M Output 164k Cache 53.9M
3 changes with usage
foundation-navigat…
contact-form
content-pages
0 4.5M 9.0M 13.5M 18.0M
LLM Call Log
19 calls
Time ▼ Phase Purpose Model Change Source Input Output Cache Duration
07:21:35 AM Other Köszönj és add meg a rövi... sonnet - session 33k 2 - 6.7s
01:27:45 AM Verification Spec Verify default content-pages event 1.0M 8k 986k 2m46s
01:24:59 AM Review gate Review opus content-pages event 50k 3k 11k 1m2s
01:13:34 AM Verification Spec Verify default contact-form event 1.8M 5k 1.8M 2m52s
01:10:42 AM Review gate Review opus contact-form event 48k 3k 11k 1m4s
12:59:41 AM Classification Classify sonnet - event 33k 993 11k 14.1s
12:59:08 AM Verification Spec Verify default foundation-navigat… event 1.0M 8k 943k 2m46s

5. Quality Gates

Deterministic quality assurance

Why isn't LLM code review enough?

Early experiments (CraftBrew run #3):

- LLM-based code review gate
- Agents "gamed" the reviewer (verbose comments = good code)
- A `TODO: implement later` slipped through and broke checkout
- Inconsistent pass/fail decisions

The solution:

- **Test, Build, E2E** = exit code 0 or not = **deterministic**
- A failing Jest test is unambiguous — you can't argue with it
- A `next build` returning exit code 1 doesn't lie
- Only spec coverage stayed LLM-based (with explicit PASS/FAIL parsing)

Gate Profiles — customization

Not every change needs every gate:

Change type	Test	Build	E2E	Review	Coverage
Feature	HARD	HARD	HARD	HARD	HARD
Bugfix	HARD	HARD	SOFT	HARD	SOFT
Cleanup	SOFT	HARD	SKIP	SOFT	SKIP
Config	SKIP	HARD	SKIP	SOFT	SKIP

- **HARD** — fail = block merge
- **SOFT** — fail = warning, doesn't block
- **SKIP** — doesn't run

> The profile system is extensible via the `ProjectType` ABC

Playwright E2E tests

Search tests

All 54 | Passed 47 | Failed 7 | Flaky 0 | Skipped 7

3/28/2026, 1:15:50 PM Total time: 10.4s

app-screenshots.spec.ts

✕	**Consumer app screenshots › app — home** app-screenshots.spec.ts:125	639ms
✕	**Consumer app screenshots › app — products** app-screenshots.spec.ts:125	597ms
✕	**Consumer app screenshots › app — product-detail** app-screenshots.spec.ts:125	167ms
✕	**Consumer app screenshots › app — cart** app-screenshots.spec.ts:125	384ms
✕	**Consumer app screenshots › app — admin-login** app-screenshots.spec.ts:125	500ms
✕	**Consumer app screenshots › app — admin-dashboard** app-screenshots.spec.ts:125	506ms
✕	**Consumer app screenshots › app — admin-products** app-screenshots.spec.ts:125	476ms

change-actions.spec.ts

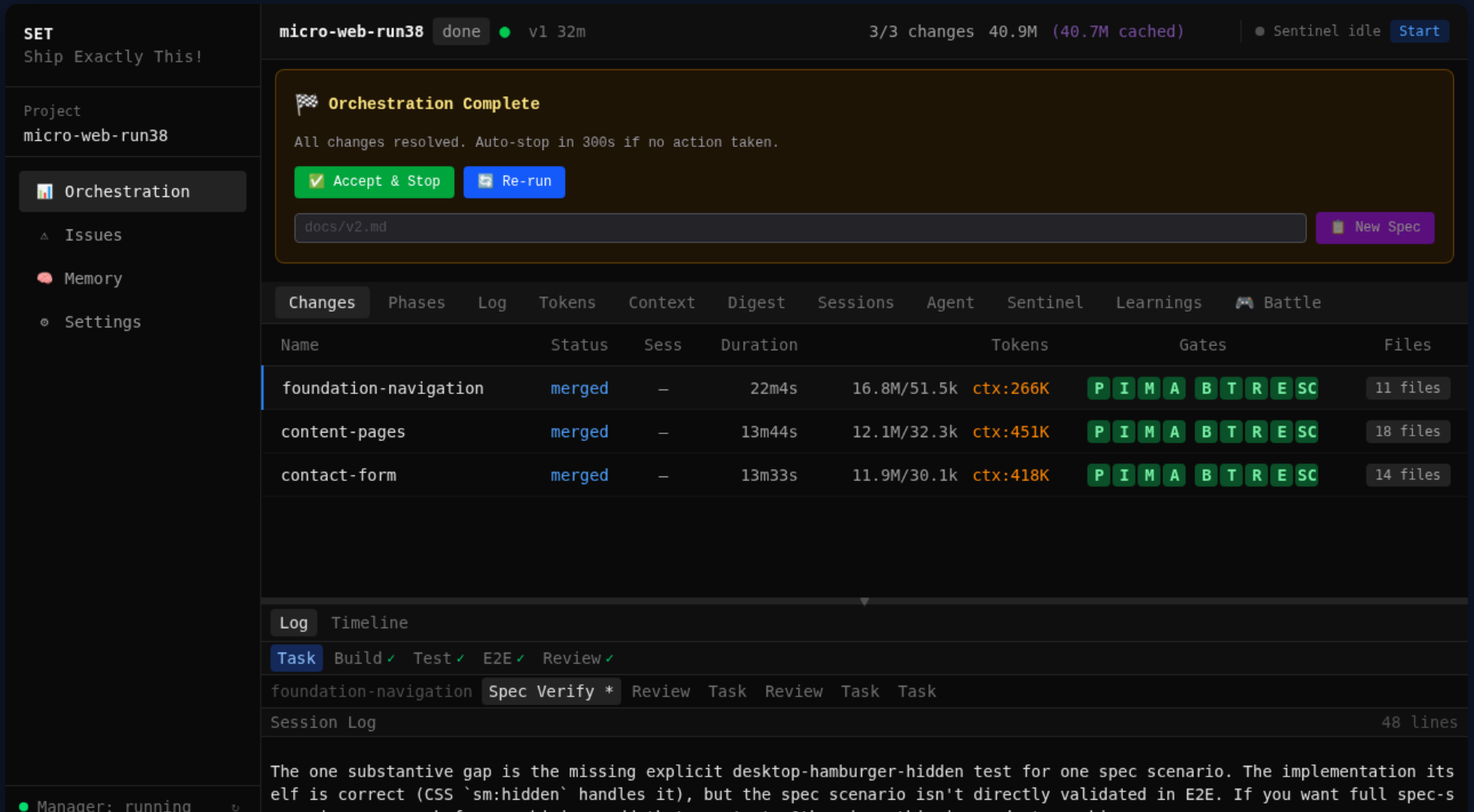
✓	**merged change has no action buttons** change-actions.spec.ts:8	1.1s

changes-data.spec.ts

6. Dashboard & Monitoring

Real-time visibility into everything

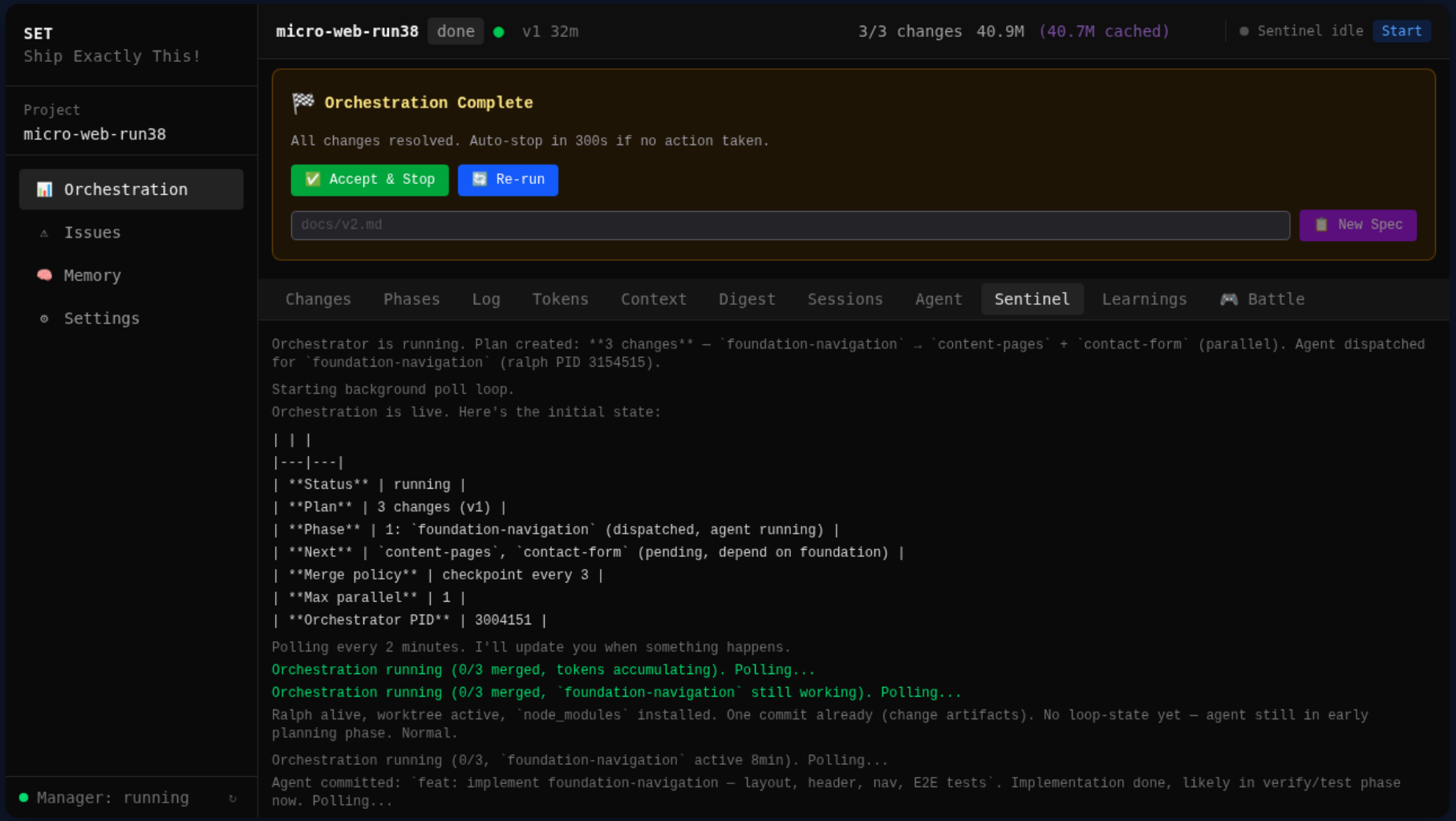
Web Dashboard — Overview



Dashboard tabs

Tab	What it shows
Changes	Status of all changes, phase, merge order
Phases	Dependency tree, gate badges (B/T/E/R/V)
Tokens	Per-change token usage chart
Sessions	Agent session list, duration, tokens
Sentinel	Supervisor decisions, restart, stall
Log	Raw orchestrator output, searchable
Agent	Live agent terminal
Learnings	Lessons, gate failures
Digest	Requirements + coverage

Sentinel supervision



7. Architecture

3 layers, extensible plugin system

3-layer architecture

```
Layer 3: External Plugins (separate repos)
 └─ e.g., set-project-fintech (IDOR, PCI compliance)

Layer 2: Built-in Modules (modules/)
 ├─ web/     → WebProjectType: Next.js, Prisma, Playwright
 └─ example/ → DungeonProjectType: reference implementation

Layer 1: Core (lib/set_orch/)
 ├─ engine.py      → State machine, orchestration
 ├─ dispatcher.py  → Worktree creation, agent dispatch
 ├─ merger.py      → Merge queue, integration gates
 ├─ verifier.py    → Quality gates (test/build/E2E/...)
 ├─ planner.py     → Spec decomposition, DAG generation
 └─ digest.py      → Requirement extraction
```

Project size

```
set-core/
├── lib/set_orch/     59K Python LOC    ← Core engine
├── modules/web/       5K Python LOC    ← Web plugin
├── mcp-server/       30K LOC           ← MCP server
├── web/              14K TypeScript    ← Dashboard
├── bin/              15K Shell         ← 57 CLI tools
├── openspec/specs/   23K LOC           ← 376 specifications
├── docs/             22K LOC           ← Documentation
└── templates/core/    3K LOC           ← Deployed rules
                      ────
                      134K LOC total
```

Statistic	Value
Development time	1,000+ hours
Commits	1,500+ (~17/day)

8. Lessons learned

100+ runs, real production experience

8 lessons from production

1. Agents need structure, not just prompts

- OpenSpec artifacts (proposal → spec → tasks) keep them focused
- Without it: 3 agents, 3 different table libraries

2. Quality gates must be deterministic

- Exit code > LLM judgment
- Agents gamed the LLM review

3. Merge conflicts are the #1 cascading failure

- Phase ordering + dependency DAG + sequential merge queue
- Cross-cutting files: Prisma schema, i18n bundle, middleware

8 lessons (continued)

5. E2E testing reveals what unit tests don't

- Stale lock files, zombie worktrees, race condition in poll cycle
- Token counter overflow at 10M+ cache tokens

6. Stall detection needs grace periods

- `pnpm install` >60s without stdout → watchdog killed it
- Context-aware timeouts: install=120s, codegen=90s, MCP=60s

7. The Sentinel pays for itself

- 3 overnight runs lost to crashes before the sentinel
- 5-10 LLM calls per run, saves hours

Scaling: MiniShop vs CraftBrew

Metric	MiniShop	CraftBrew #7	Multiplier
Changes	6	15	2.5x
Source files	47	150+	3x
DB models	~8	28	3.5x
Merge conflicts	0	4 (all resolved)	--
Human intervention	0	0	--
Wall time	1h 45m	~6h	3.4x
Tokens	2.7M	~11M	**4x**

> Token scaling is **super-linear** (4x tokens for 2.5x changes) — later changes need more context.

Convergence — measuring reproducibility

Comparing two independent MiniShop runs:

Dimension	Match
DB schema (models, fields, relations)	**100%**
Conventions (naming, structure)	**100%**
Routes (URLs, API endpoints)	**83%**
Overall	**83/100**

Context for the 83% score:

- The remaining 17% is **stylistic**, not structural

9. Roadmap

Where we're heading

Development priorities

Direction	Goal	Status
Divergence reduction	Template optimization, scaffold testing	Measurable improvement on simple projects
Build time optimization	Parallel gates, incremental build, cache	Currently sequential; researching
Session context reuse	Reuse context across iterations	Reduce cold-start token overhead
Memory optimization	Relevance scoring, dedup, auto-rule conversion	Dedup + consolidation operational
Gate intelligence	Adaptive thresholds based on historical pass rates	Gate profiles operational
Merge conflict	Proactive cross-cutting file detection	Phase ordering operational

Bigger plans

Core/Web separation

- 170+ web-specific references leaked into the core
- Move to `modules/web/` via new `ProjectType` ABC methods
- Goal: external plugins shouldn't depend on web logic

NIS2 Compliance Layer

- EU 2022/2555 directive support
- Template rules, verification rules, dedicated gate
- Opt-in via configuration flag

shadcn/ui Design Connector

- `components.json`, `tailwind.config.ts`, `globals.css` parsing

The full ecosystem

Repository	Description
set-core	Core engine, web module, dashboard, CLI
set-voice-agent-delivery	Voice-based agent delivery

Tech stack:

Component	Technology
Agent runtime	Claude Code (Anthropic)
Workflow	OpenSpec
Isolation	Git worktrees
Engine	Python + FastAPI + uvicorn
Dashboard	Next.js + React + Tailwind CSS

Summary

SPECIFY → DECOMPOSE → EXECUTE → SUPERVISE → VERIFY → LEARN

6 changes | 1h 45m | 0 interventions | 70 tests | 100% spec coverage

Greenfield, brownfield, isolated unit — same pipeline.

Questions?

Resources:

- GitHub: `github.com/tatargabor/set-core`
- Web: `setcode.dev`
- Benchmarks: `docs/learn/benchmarks.md`
- Lessons learned: `docs/learn/lessons-learned.md`

Try it:

```
# Install
pip install -e .
pip install -e modules/web

# First run
./tests/e2e/runners/run-micro-web.sh
```